

Ciência de Dados

Prof. Túlio Ribeiro

Núcleo de Ciência de Dados e Inteligência Artificial
Universidade de Fortaleza



Unifor

Redução de Dimensionalidade

O que é Dimensionalidade?

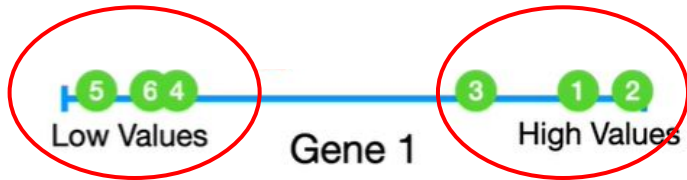
Conceito: Na computação e estatística, a dimensionalidade de um dataset refere-se ao número de variáveis de entrada ou atributos (features) que descrevem um objeto. Se você tem uma tabela com altura, peso, idade e pressão arterial de pacientes, você está trabalhando em um espaço quadridimensional (4D).

Representação: Cada linha do dado é um vetor em um espaço de d dimensões.

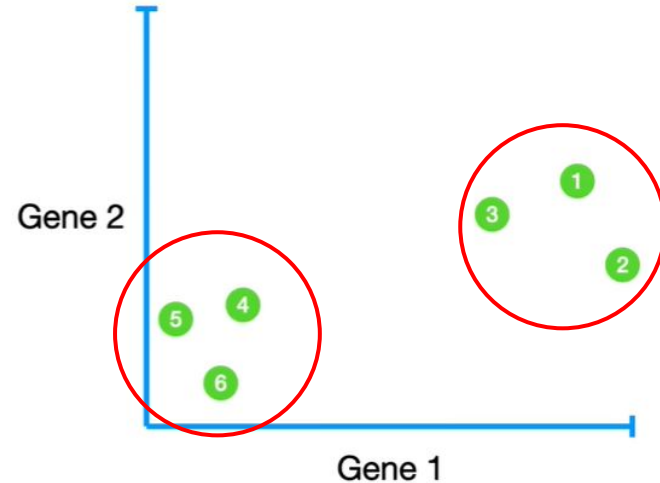
Problema: Visualizar em 2D ou 3D é simples, mas dados reais (genômica, imagens, finanças) possuem milhares de dimensões. Reduzir a dimensionalidade é o processo de destilar essas variáveis para encontrar os padrões essenciais ocultos, eliminando redundâncias (ex: se duas variáveis medem quase a mesma coisa).

O que é Dimensionalidade?

	Mouse 1	Mouse 2	Mouse 3	Mouse 4	Mouse 5	Mouse 6
Gene 1	10	11	8	3	1	2



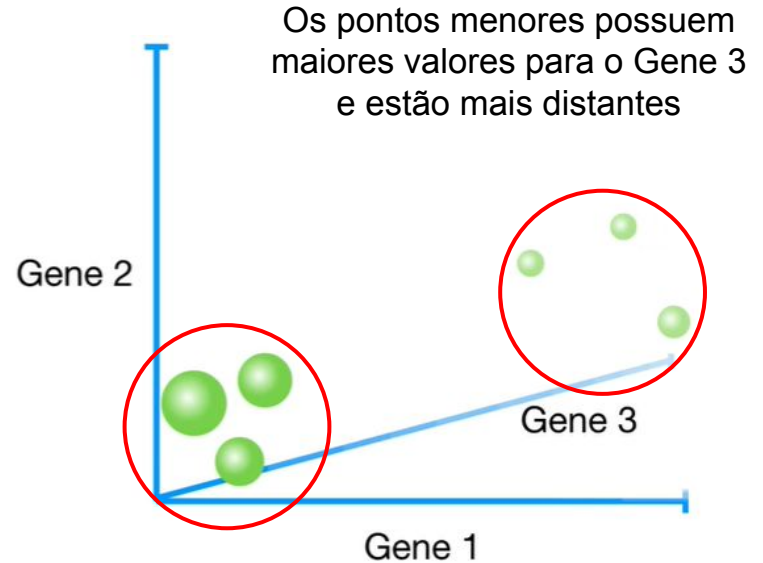
	Mouse 1	Mouse 2	Mouse 3	Mouse 4	Mouse 5	Mouse 6
Gene 1	10	11	8	3	1	2
Gene 2	6	4	5	3	2.8	1



O que é Dimensionalidade?



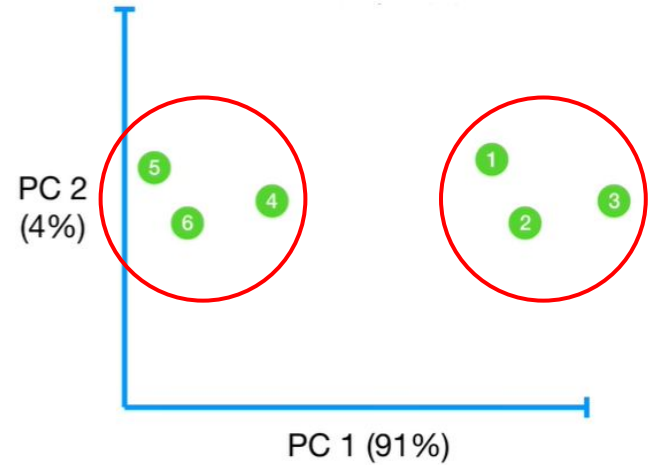
	Mouse 1	Mouse 2	Mouse 3	Mouse 4	Mouse 5	Mouse 6
Gene 1	10	11	8	3	2	1
Gene 2	6	4	5	3	2.8	1
Gene 3	12	9	10	2.5	1.3	2



O que é Dimensionalidade?



	Mouse 1	Mouse 2	Mouse 3	Mouse 4	Mouse 5	Mouse 6
Gene 1	10	11	8	3	2	1
Gene 2	6	4	5	3	2.8	1
Gene 3	12	9	10	2.5	1.3	2
Gene 4	5	7	6	2	4	7



A Maldição da Dimensionalidade

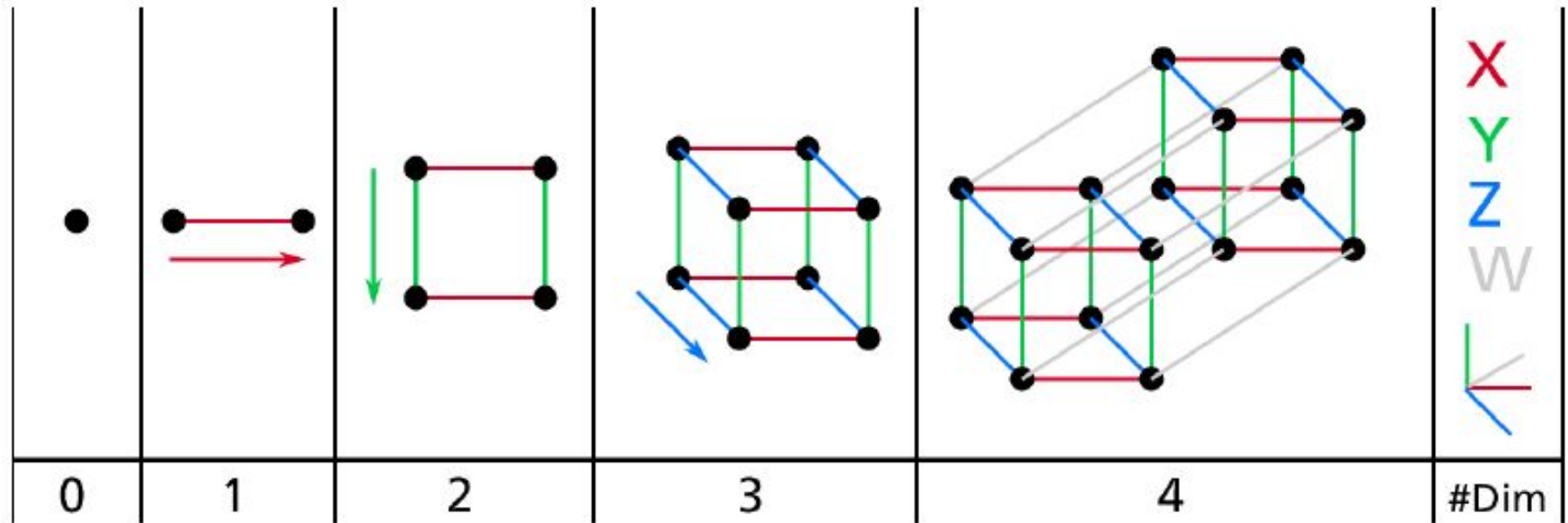
Esparsidade dos Dados: Oculta padrões e exige volumes massivos de amostras para ter validade estatística.

Complexidade e Custo: Exige alto poder computacional e dificulta a interpretação humana.

Aplicações da Redução de Dimensionalidade

- **Redução de Ruído:** Filtragem de variáveis irrelevantes.
- **Visualização de Dados:** Projeção de dimensões complexas em 2D ou 3D.
- **Análise de Cluster:** Destaque para as características essenciais para melhorar os agrupamentos.
- **Pré-processamento:** Acelera e otimiza o treino de modelos de Machine Learning.

A Maldição da Dimensionalidade



Ponto, segmento, quadrado, cubo e tesseract (hipercubos de 0D a 4D)

SVD – Decomposição em Valores Singulares

SVD – Decomposição em Valores Singulares

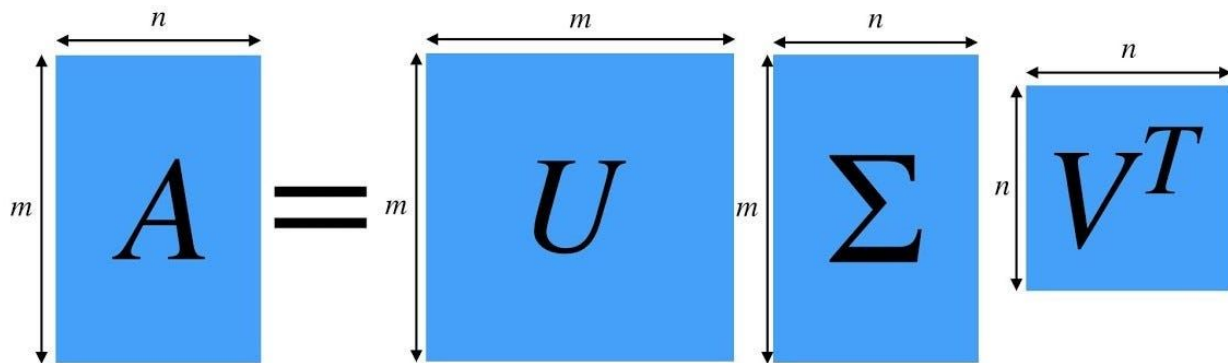
É uma técnica de álgebra linear que fatora qualquer matriz de dados em componentes ortogonais independentes. É o “motor” matemático por trás de várias técnicas de compressão.

$$A = U\Sigma V^T$$

- A : Matriz original de dados com dimensões $m \times n$ (ex: m amostras e n features).
- U : Matriz ortogonal contendo os autovetores esquerdos. Eles representam como as amostras (linhas) se relacionam entre si.
- Σ (Sigma): Matriz diagonal contendo os “valores singulares”. Eles representam a “força” ou importância de cada componente para reconstruir a matriz original.
- V^T : Matriz ortogonal transposta contendo os autovetores direitos. Representam como as variáveis (colunas) se relacionam entre si.

SVD – Decomposição em Valores Singulares

$$A = U\Sigma V^T$$



SVD – Decomposição em Valores Singulares

O SVD identifica quais combinações de pixels são mais importantes para formar a imagem. Se descartarmos os menores valores de Σ podemos reconstruir a imagem usando muito menos dados (compressão).

Original Colorida



Original Tons de Cinza



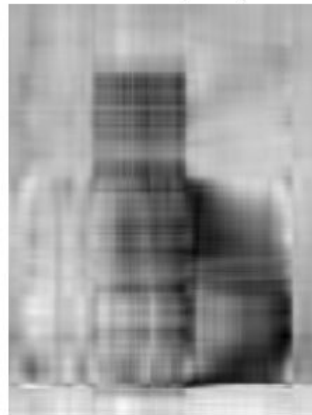
Compressão (k=50)



Compressão (k=20)



Extrema (k=5)



SVD – Decomposição em Valores Singulares

Vantagens:

- **Matematicamente garantido:** sempre encontra a melhor aproximação linear possível de uma matriz (Teorema de Eckart-Young).
- **Flexível:** aplicável a qualquer matriz (não precisa ser quadrada).

Desvantagens:

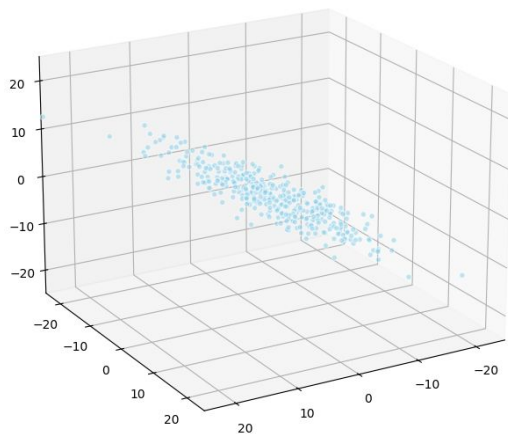
- Computacionalmente custoso para matrizes gigantescas.
- Componentes gerados (fatores latentes) são puramente matemáticos e muitas vezes impossíveis de interpretar humanamente.

PCA – Análise de Componentes Principais

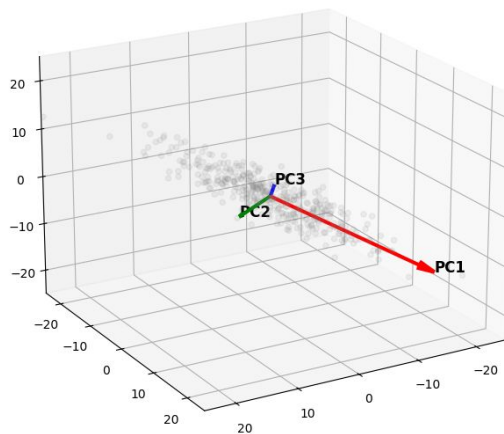
PCA – Análise de Componentes Principais

Busca novas direções (eixos) no espaço, chamadas Componentes Principais, que capturem a maior variância dos dados. O primeiro componente explica a maior parte do “espalhamento” dos dados, e os seguintes explicam o restante de forma ortogonal.

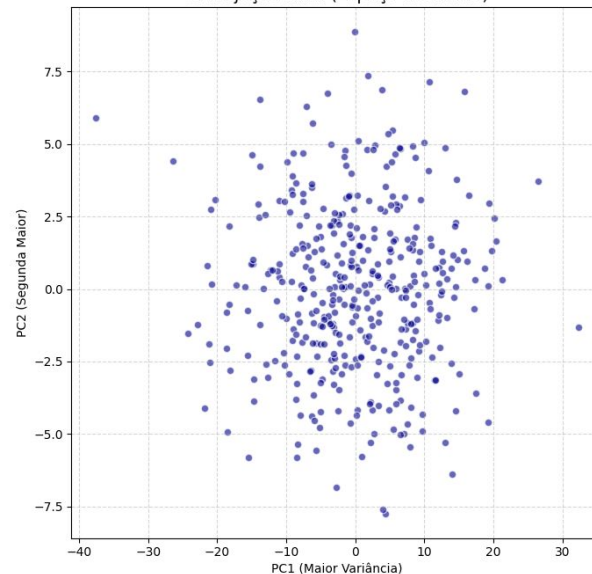
1. Dados Originais no Espaço 3D



2. Autovetores



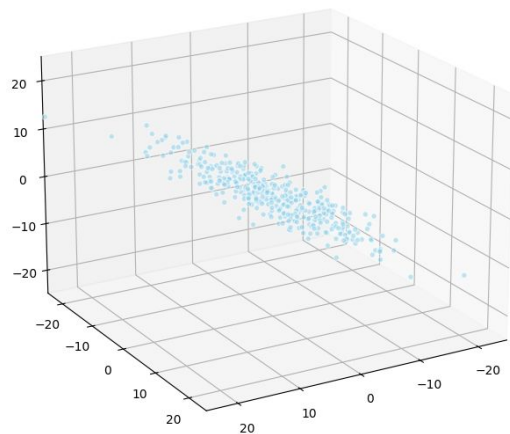
3. Projeção Final (Espaço Reduzido)



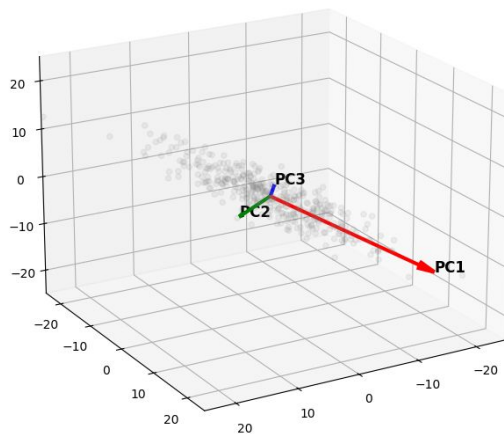
PCA – Análise de Componentes Principais

Imagine segurar uma xícara 3D e iluminá-la com uma lanterna contra a parede. O PCA gira essa xícara matematicamente até que a “sombra” dela (a projeção 2D) seja a maior e mais detalhada possível, mostrando seu formato principal.

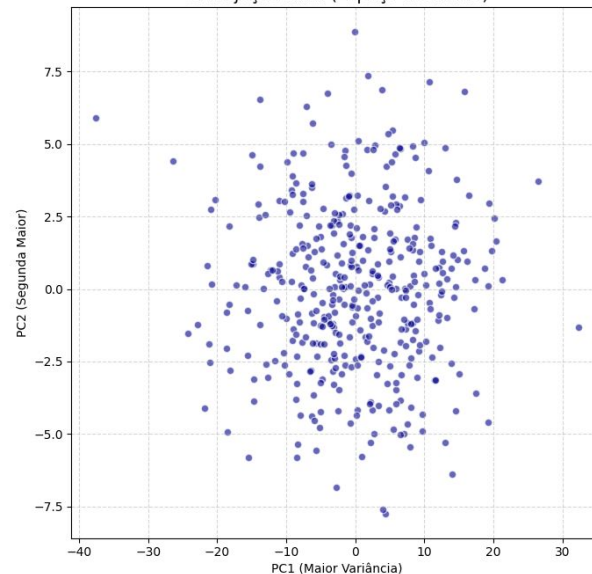
1. Dados Originais no Espaço 3D



2. Autovetores



3. Projeção Final (Espaço Reduzido)



PCA – Análise de Componentes Principais

Fórmulas Matemáticas:

1. **Matriz de Covariância:** $C = \frac{1}{n-1}X^T X$

- C : A matriz de covariância que captura como as variáveis variam juntas.
- n : O número total de amostras (linhas) no conjunto de dados.
- X : A matriz de dados originais já centralizada (onde a média foi subtraída de cada coluna).
- X^T : A matriz X transposta.

2. **Autodecomposição:** $Cv = \lambda v$

- v : Os autovetores, que são as novas direções (os Componentes Principais).
- λ (Lambda): Os autovalores, que indicam a quantidade de variância (informação) que aquele eixo v específico capturou.

PCA – Análise de Componentes Principais

Passo a Passo (Projeção)

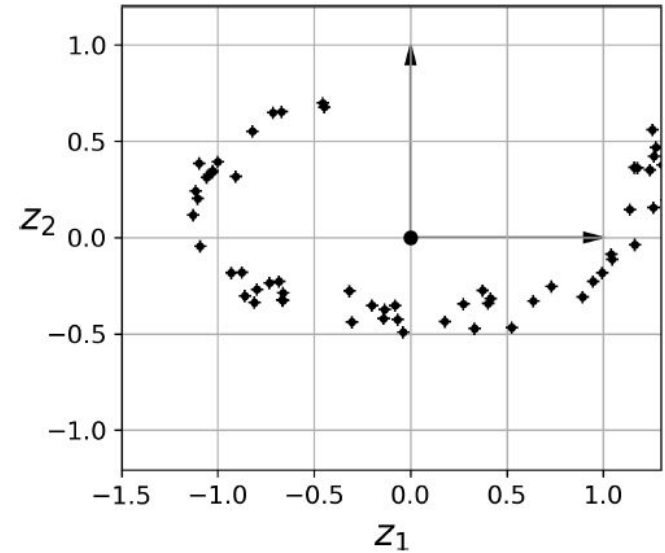
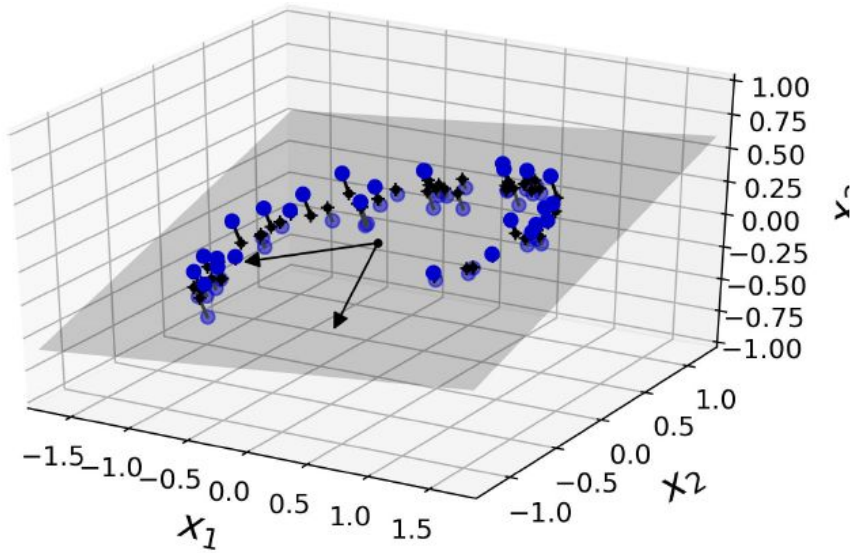
1. **Centralização:** Subtrair a média para levar os dados à origem.
2. **Covariância:** Calcular C para medir redundâncias.
3. **Autodecomposição:** Resolver $Cv = \lambda v$.
 - a. v (Autovetores): Direção dos novos eixos.
 - b. λ (Autovalores): Quanta variância cada eixo explica.
4. **Projeção:** Multiplicar os dados originais pelos autovetores escolhidos ($Y = X \cdot W$).
 - a. **Ideia:** Imagine a “sombra” de um objeto 3D em uma folha 2D. O PCA rotaciona o objeto para que a sombra seja a mais detalhada possível.

PCA – Análise de Componentes Principais

PCA via SVD

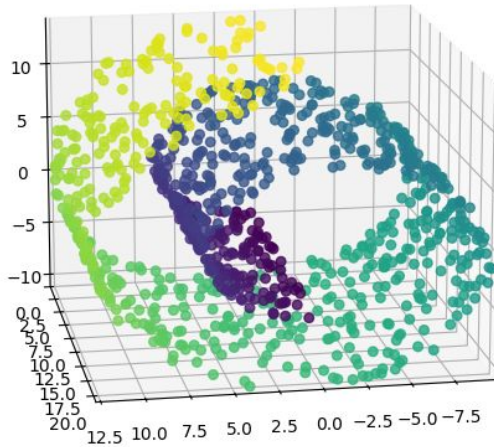
- **Problema do PCA Clássico:** Calcular a matriz de covariância ($X^T X$) para datasets com milhares de dimensões é computacionalmente lento e propenso a erros de arredondamento.
- **Solução via SVD:** Subtraímos a média (centralizar) e aplicamos o SVD ($X = U \Sigma V^T$) na nossa matriz de dados X .
- **Resultado:** As colunas da matriz V (autovetores direitos do SVD) são exatamente os mesmos Componentes Principais do PCA.
- **Na Prática:** Bibliotecas como o scikit-learn no Python não calculam a covariância para fazer o PCA; elas aplicam o algoritmo do SVD por baixo dos panos por ser muito mais rápido e estável!

PCA – Análise de Componentes Principais

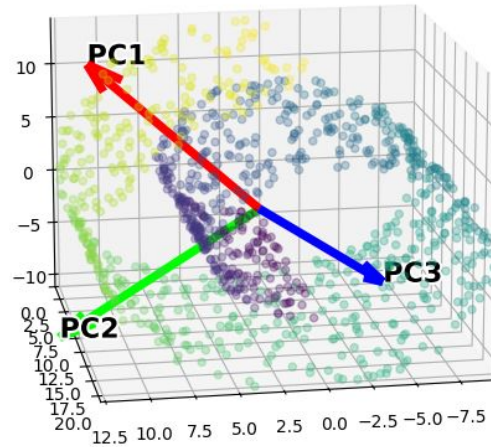


PCA – Análise de Componentes Principais

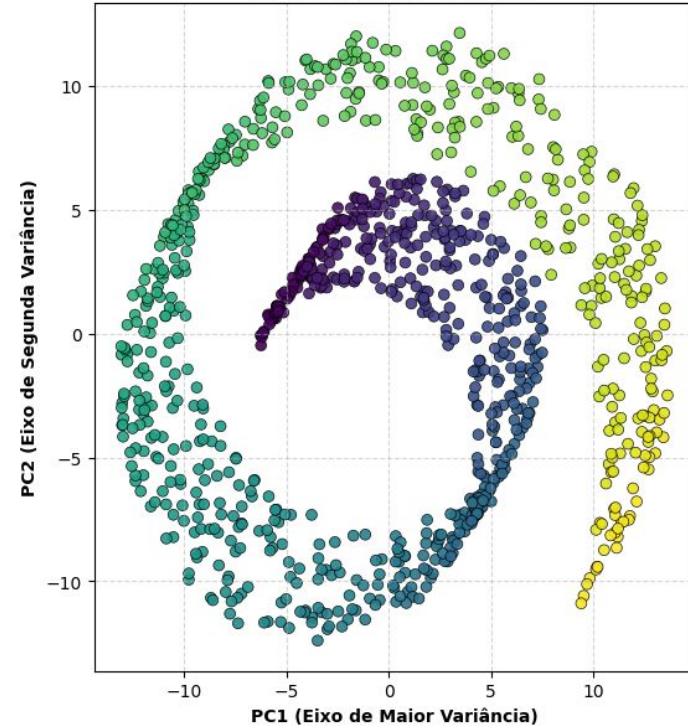
1. Espaço Original (Swiss Roll)



2. Identificação dos Autovetores



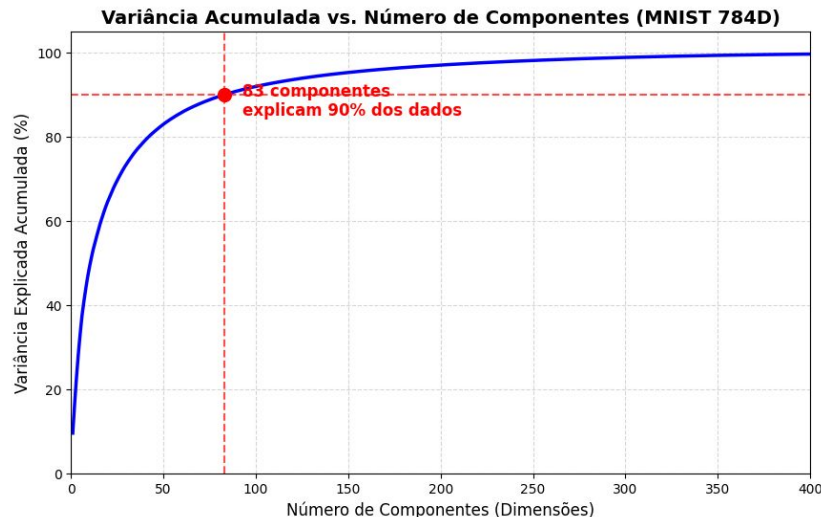
3. Projeção PCA (2D)



PCA – Análise de Componentes Principais

Variância Explicada: Indica quanta “informação” (variabilidade) do dataset original cada Componente Principal (PC) conseguiu capturar.

- Onde o gráfico forma um “cotovelo” (o ganho de informação diminui drasticamente).
- **Limiar de Variância:** 70% a 95% de variância acumulada.



PCA – Análise de Componentes Principais

Vantagens:

- **Determinístico:** o resultado é sempre exatamente o mesmo.
- **Ortogonalidade:** garante que não há redundância de informação entre os novos eixos.
- Extremamente veloz.

Desvantagens:

- **Linearidade:** falha em dados curvos, espirais ou complexos.
- Sensibilidade extrema a outliers (dados anômalos puxam o eixo de variância).

LDA - Análise Discriminante Linear

LDA - Análise Discriminante Linear

Diferente do PCA, o LDA é supervisionado (usa os rótulos das classes). Ele projeta os dados de forma que pontos da mesma classe fiquem muito juntos, e pontos de classes diferentes fiquem o mais longe possível.

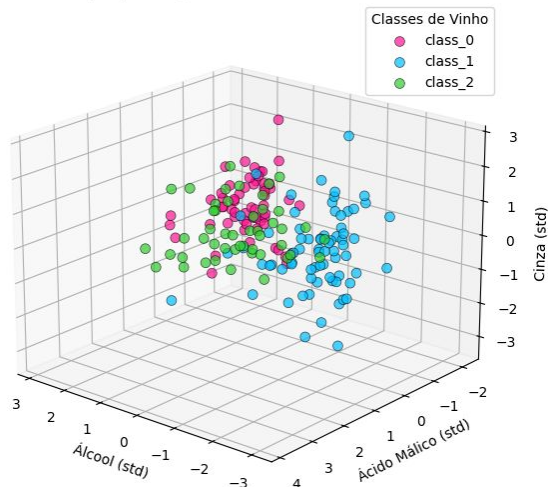
Fórmula: $J(w) = \frac{w^T S_B w}{w^T S_W w}$

- $J(w)$: A função objetivo que o algoritmo tenta maximizar para encontrar a melhor separação.
- w : O vetor de projeção (o novo eixo ou dimensão para onde os dados irão).
- S_B (Between-class): Matriz de dispersão entre as classes. Mede o quão distantes estão os centros (médias) de cada classe diferente.
- S_W (Within-class): Matriz de dispersão dentro das classes. Mede o quão espalhados estão os dados em torno do centro da sua própria classe.

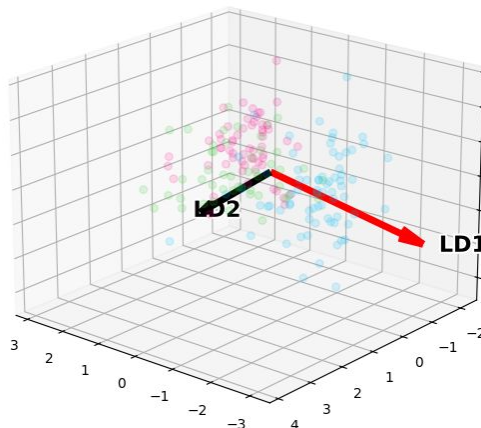
LDA - Análise Discriminante Linear

Diferente do PCA, o LDA não quer a “maior sombra”. Ele gira a lanterna propositalmente buscando o único ângulo onde a sombra do Vinho Tinto fique completamente longe da sombra do Vinho Branco. A separação é mais importante que o formato.

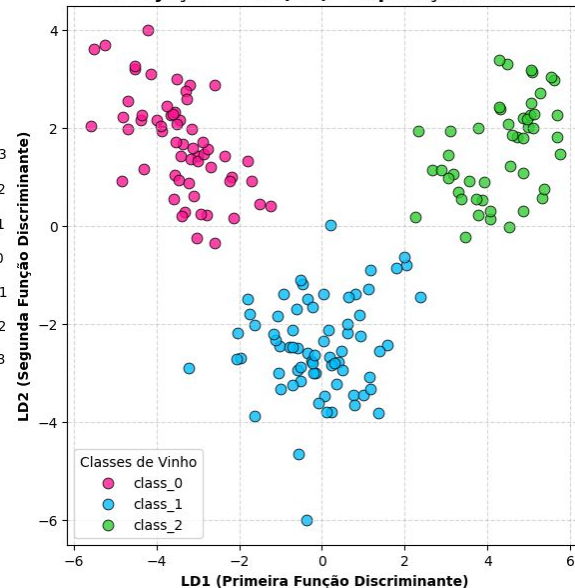
1. Espaço Original (3 de 13 Atributos)



2. Vetores Discriminantes (LDA 3D)



3. Projeção Final (2D) - Separação Máxima



LDA - Análise Discriminante Linear

Vantagens:

- **Supervisionado:** usa os rótulos ativamente para forçar a separação das classes.
- É o pré-processamento ideal antes de modelos de Classificação.

Desvantagens:

- Exige dados previamente classificados (rotulados).
- A redução máxima é limitada (N° de classes - 1). Ex: Separar gatos e cachorros reduz o dado a apenas 1 dimensão.

t-SNE – t-distributed Stochastic Neighbor Embedding

t-SNE – t-distributed Stochastic Neighbor Embedding

Foca em manter “vizinhos próximos” juntos. Calcula a probabilidade de dois pontos serem vizinhos no espaço original e tenta recriar essas mesmas probabilidades no espaço reduzido (2D ou 3D).

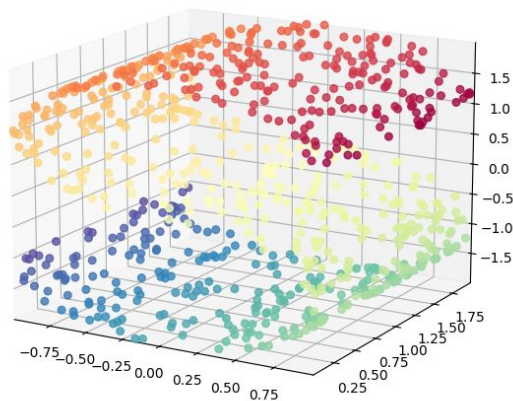
Fórmula Matemática: Minimização da Divergência Kullback-Leibler: $C = \sum_i \sum_j p_{j|i} \log \frac{p_{j|i}}{q_{j|i}}$

- C : Função de custo (o erro total que o algoritmo tenta reduzir a zero ajustando os pontos).
- $p_{j|i}$: Probabilidade do ponto j ser vizinho do ponto i no espaço de alta dimensão (original), calculada usando uma distribuição Gaussiana normal.
- $q_{j|i}$: Probabilidade do ponto j ser vizinho do ponto i no espaço de baixa dimensão (reduzido), calculada usando uma distribuição t-Student. A distribuição t-Student é usada aqui porque ela tem “caudas mais pesadas”, o que evita que pontos diferentes fiquem amontoados uns sobre os outros no gráfico final.

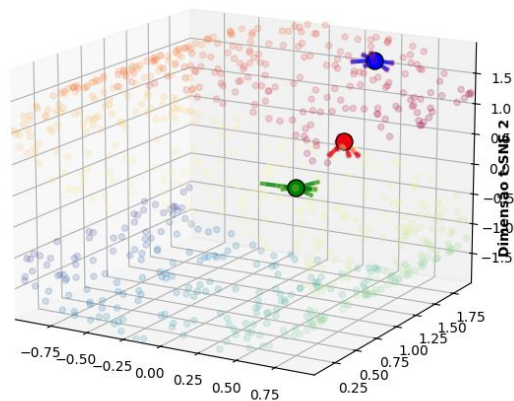
t-SNE – t-distributed Stochastic Neighbor Embedding

O t-SNE não usa linhas retas. Ele enxerga os dados como pessoas de mãos dadas formando um “S”. Ele então desenrola esse “S” amassando e esticando o espaço, com uma única regra estrita: ninguém pode soltar a mão de seu vizinho mais próximo.

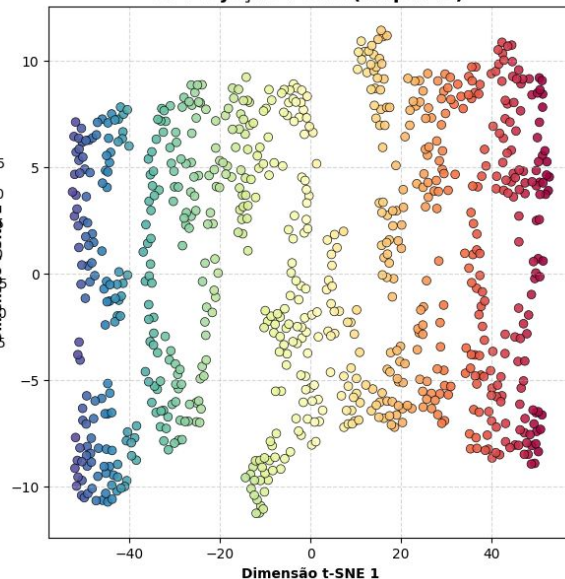
1. Espaço Original (Curva em S)



2. O Cálculo: Probabilidade de Vizinhança



3. Projeção t-SNE (Mapa 2D)



t-SNE – t-distributed Stochastic Neighbor Embedding

Por que o “t” no t-SNE?

Problema do Amontoamento (Crowding): Em altas dimensões, os pontos têm muito “espaço” para se afastar. Ao projetar em 2D (espaço pequeno), eles tendem a colapsar uns sobre os outros.

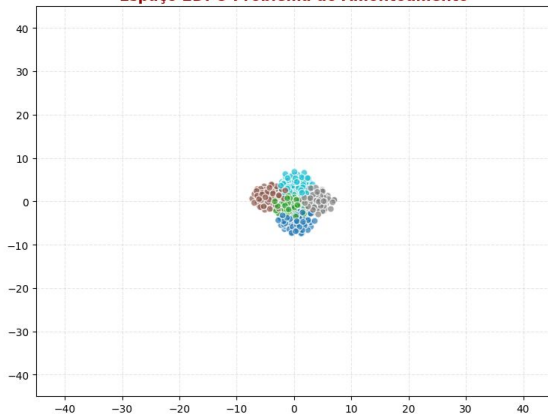
Distribuição Gaussiana: Decai muito rápido. Pontos moderadamente distantes ficam com probabilidade quase zero, sendo “esmagados” no centro do gráfico.

Distribuição t-Student: Possui “caudas longas”. Isso permite que pontos que estão longe no espaço original continuem distantes na visualização 2D.

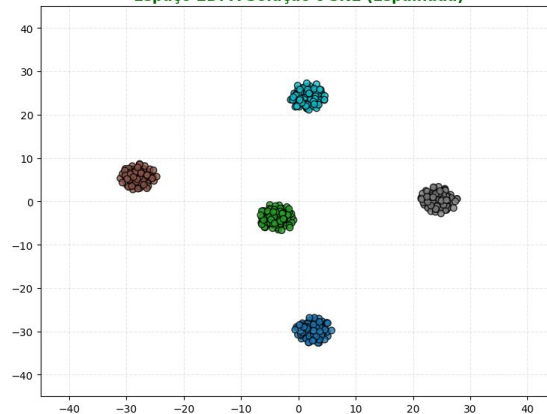
Resultado: Agrupamentos (clusters) muito mais nítidos e visualmente separáveis.

t-SNE – t-distributed Stochastic Neighbor Embedding

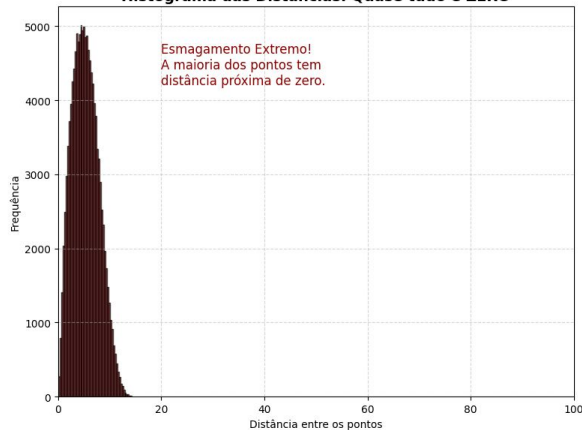
Espaço 2D: O Problema do Amontoamento



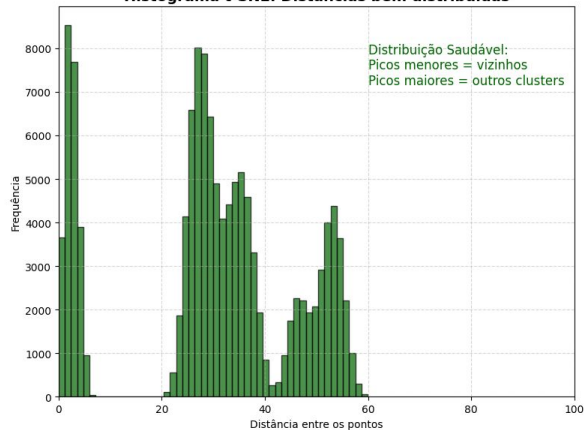
Espaço 2D: A Solução t-SNE (Espalhada)



Histograma das Distâncias: Quase tudo é ZERO



Histograma t-SNE: Distâncias bem distribuídas



t-SNE – t-distributed Stochastic Neighbor Embedding

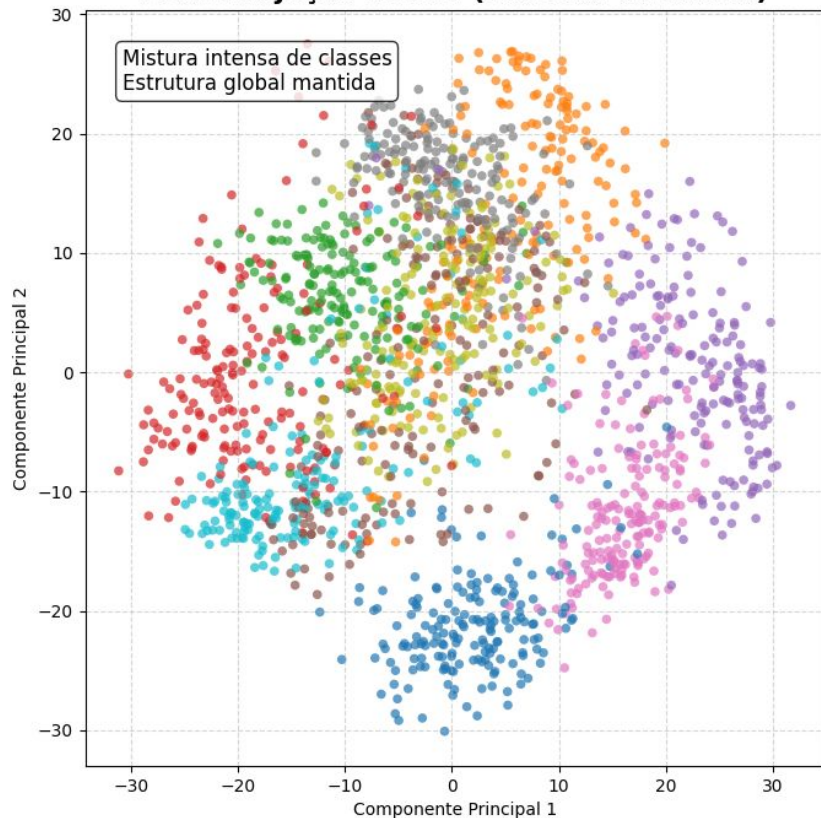
A perplexidade é o hiperparâmetro central do t-SNE. Ela define o tamanho da vizinhança (número estimado de vizinhos próximos), ajustando matematicamente a variância da distribuição Gaussiana original de alta dimensão. Valores típicos variam de 5 a 50.

- **Baixa (ex: 2 a 5):** Foco extremo no micro-local. Fragmenta os dados em centenas de pequenos falsos agrupamentos, isolando ruídos.
- **Alta (ex: próxima ao total de amostras):** Trata o dataset inteiro como uma única vizinhança. Os pontos colapsam em uma massa homogênea, destruindo a estrutura real.

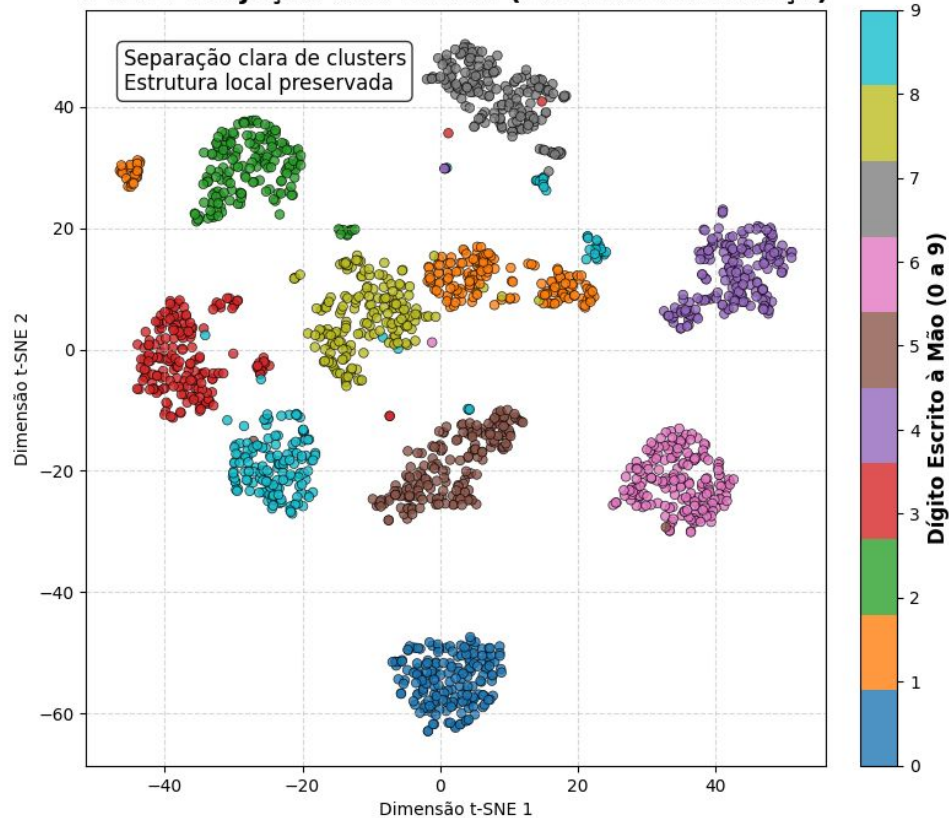
Deve ser estritamente menor que o total de amostras. Exige testes com múltiplos valores para validar se os clusters encontrados são reais ou apenas artefatos visuais paramétricos.

t-SNE – t-distributed Stochastic Neighbor Embedding

PCA: Projeção Linear (Foco na Variância)



t-SNE: Projeção Não-Linear (Foco na Vizinhança)



t-SNE – t-distributed Stochastic Neighbor Embedding

Vantagens:

- Extraordinário para abrir clusters (subgrupos) em dados hipercomplexos.
- Resolve o “amontoamento” visual de dados graças à distribuição t-Student.

Desvantagens:

- **Probabilístico:** cada vez que você executa, o gráfico sai numa rotação ou formato diferente (precisa fixar a semente).
- Lento para grandes volumes de dados ($O(n^2)$).
- Não preserva a “geografia” global (o espaço vazio entre ilhas de dados no gráfico não significa nada).

UMAP – Uniform Manifold Approximation and Projection

UMAP – Uniform Manifold Approximation and Projection

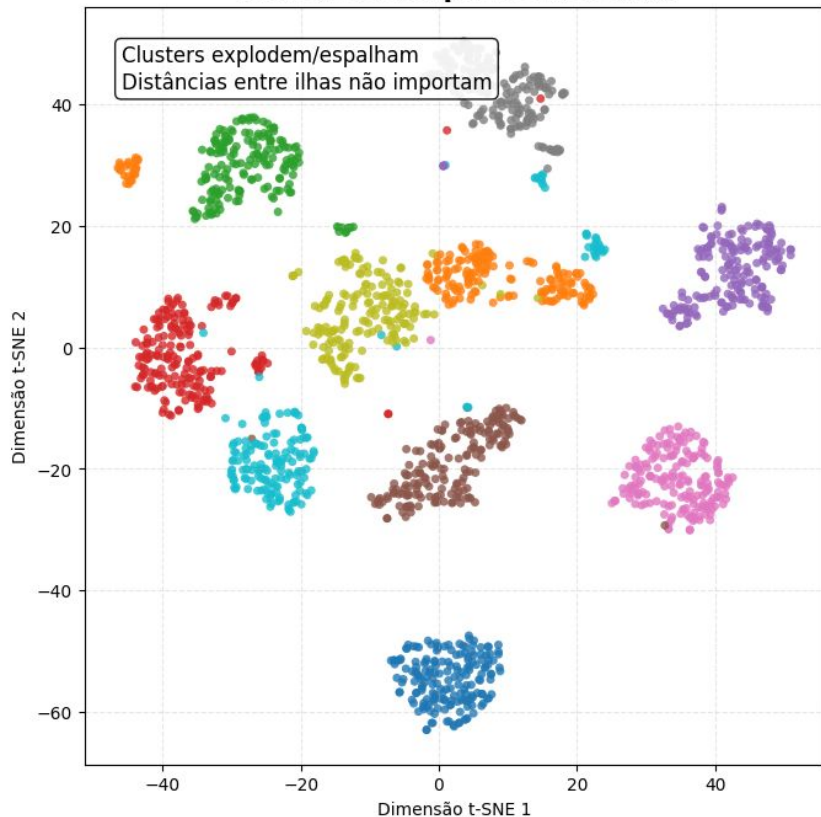
A evolução do t-SNE: Usa conceitos de geometria e topologia algébrica para criar um grafo tridimensional (simplicial sets) dos dados e depois projeta esse grafo em uma dimensão menor.

Complexidade Algorítmica: O custo é empiricamente próximo a $O(n^{1.14})$, tornando-o mais rápido que o t-SNE para datasets maiores.

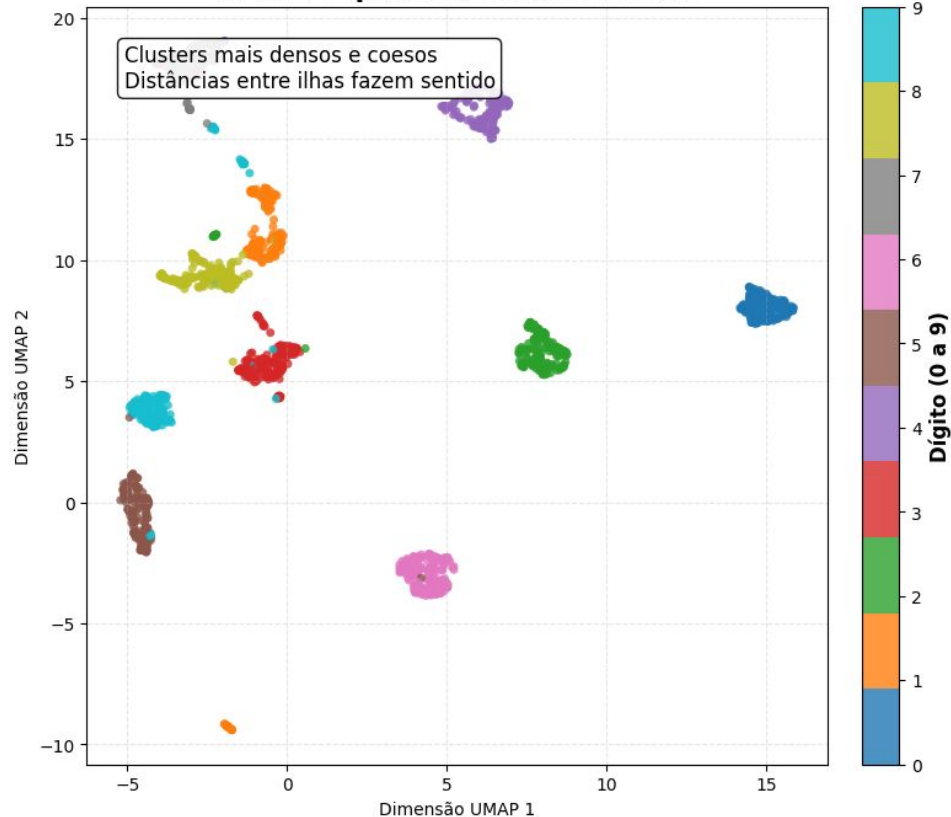
O t-SNE desenha os bairros perfeitos da cidade, mas os joga aleatoriamente no mapa. O UMAP, usando matemática avançada, não apenas desenha os bairros muito bem, mas os posiciona no mapa respeitando a distância correta de uma cidade para a outra.

UMAP – Uniform Manifold Approximation and Projection

t-SNE: Foco Apenas no Local



UMAP: Equilíbrio Local e Global



UMAP – Uniform Manifold Approximation and Projection

Vantagens:

- O “Estado da Arte”: muito mais rápido e escalável que o t-SNE.
- Preserva a vizinhança local e a topologia global (distâncias fazem sentido).
- Permite projetar dados novos em um mapa já treinado.

Desvantagens:

- Base matemática altamente árida e densa (Geometria Riemanniana).
- Sensível à má calibração de parâmetros (`n_neighbors`).

Autoencoders – Redução via Redes Neurais

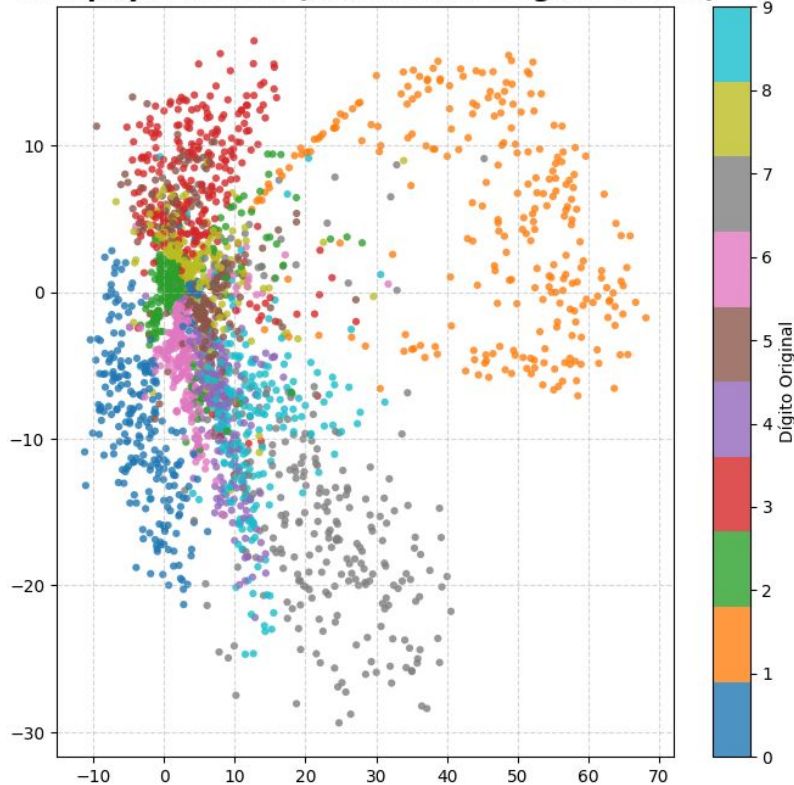
Autoencoders – Redução via Redes Neurais

É uma rede neural profunda treinada para copiar sua própria entrada para sua saída. O segredo está no meio da rede, onde há uma camada oculta com poucos neurônios (o “gargalo”). A rede é forçada a comprimir o dado para passar por esse gargalo.

Pedimos à rede para resumir um livro de 300 páginas em apenas 2 páginas (o “Gargalo”). Depois, sem olhar o original, o Decoder tenta reescrever o livro lendo só as 2 páginas. Se o dígito (imagem) saiu perfeitamente reconstruído no final, essas 2 coordenadas mapearam a alma do dado.

Autoencoders – Redução via Redes Neurais

1. Espaço Latente (A Visão do Gargalo em 2D)



2. Original vs. Reconstrução a partir de 2 dimensões

Original
(784D)



Reconstruído
(De 2D)



Autoencoders – Redução via Redes Neurais

Vantagens:

- Aprende qualquer padrão não-linear através de funções de ativação.
- Pode ser arquitetado para aceitar imagens brutas (CNNs) e limpar ruídos.

Desvantagens:

- A rede pode “decorar” (overfitting) em vez de extrair a essência do dado.
- Exige tentativa e erro para projetar o gargalo e alto custo de GPU.

Fim